Online Code Compiler

Dhadbanjan Sai Adarsh
Computer Science and Engineering
Institute of Aeronautical Engineering
Email: adarshsai619@gmail.com

Bacchewal Sankeerth
Computer Science and Engineering
Institute of Aeronautical Engineering
Email: bunnybacchewal@gmail.com

***Abstract*—In the modern software development landscape, accessibility and ease of use are key to fostering learning and productivity. This paper presents the design and implementation of an Online Code Compiler that allows users to write, execute, and manage programming projects directly in their web browser. The system integrates the Monaco Editor for code editing, Judge0 API for execution, and Xterm.js with Node-PTY for terminal emulation. In addition, a built-in file management system enables project-level file creation and organization. The platform focuses on *usability, real-time feedback, and smooth interaction* rather than performance benchmarking. Built using React, Node.js, and Express, it offers a lightweight and responsive coding experience ideal for learners and developers.**

***Index Terms*—Online Code Compiler, Web IDE, Judge0 API, React, Node.js, Express, Xterm.js, Node-PTY, File Management, Browser-based Development.**

I. Introduction

Programming has evolved beyond local Integrated Development Environments (IDEs) to web-based platforms that provide the same functionalities without installation. Traditional IDEs such as Eclipse or Visual Studio Code require complex configurations and language dependencies, which can hinder beginner learning or quick testing.

Online compilers bridge this gap by offering a ready-to-use environment that supports code execution within a browser. However, many online compilers are limited — they allow only single-file execution, lack terminal access, and provide minimal interactivity.

Our project, the **Online Code Compiler**, aims to overcome these limitations. It combines a modern code editor, real-time terminal, and project file management in a single web application. Users can:

- Write code in multiple languages.
- Execute programs using the Judge0 API.
- Use a simulated terminal for command-line operations.
- Manage files and folders within projects.

This integration provides a realistic and engaging experience that closely resembles using a desktop IDE, making it ideal for students, developers, and educators.

II. Background and Related Work

Online development environments have become an integral part of modern programming education. Platforms such as *Replit*, *CodeSandbox*, and *JDoodle* have popularized browser-based coding.

Replit offers collaborative coding but relies on a heavy backend infrastructure and user authentication. **CodeSandbox** focuses mainly on frontend frameworks such as React or Vue, limiting support for general-purpose languages. **JDoodle** supports multiple languages but provides limited interactivity and no persistent file system.

Our project differs by focusing on **usability, interactivity, and flexibility**. The integration of Xterm.js and Node-PTY allows real-time terminal-based interactions, while the addition of a built-in file manager provides users with a project-oriented workspace.

In educational settings, such usability improvements can make coding more intuitive and interactive, reducing setup time and increasing engagement.

III. System Architecture

The proposed system follows a **client-server architecture** where the frontend handles user interaction and visualization, while the backend manages execution and terminal communication.

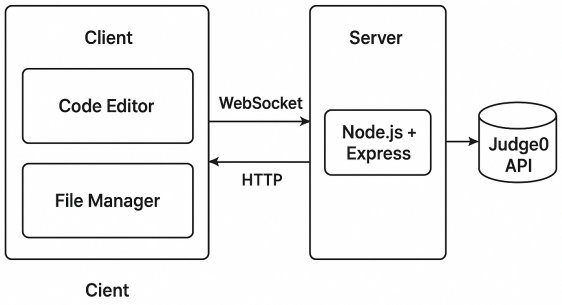

Fig. 1. System Architecture of the Online Code Compiler

A. Frontend Components

The frontend, developed using **React**, includes three major components:

- **Monaco Editor:** Provides syntax highlighting, code suggestions, and language awareness for multiple programming languages.
- **Terminal Interface:** Implemented using Xterm.js, it displays real-time shell output and accepts user commands.

- **File Manager:** A custom sidebar that allows users to create, rename, and delete files or folders in a structured project hierarchy.

The frontend communicates with the backend using REST APIs and WebSockets for asynchronous data flow and live terminal updates.

B. Backend Components

The backend is built using **Node.js** and **Express**. Its main responsibilities are:

1) Managing communication with the **Judge0 API** for remote code execution.
2) Maintaining a pseudo-terminal process using **Node-PTY**.
3) Handling user file operations through secure REST endpoints.

Each terminal session creates a dedicated Node-PTY instance, allowing multiple users to interact independently. The backend securely handles code submissions by encoding them before sending them to Judge0's sandbox environment.

C. Data Flow

Figure 1 shows the data flow. The user writes code in the Monaco Editor, selects a language, and presses "Run." The backend forwards this request to the Judge0 API, retrieves the result, and sends it back to the frontend for display. Simultaneously, the terminal provides interactive command-line feedback.

The file manager maintains the state of files on the client and mirrors changes on the server, ensuring synchronization between both ends.

IV. Implementation Details

The project is divided into frontend and backend modules that interact seamlessly.

A. Frontend Implementation

The **Monaco Editor** was embedded using the official NPM package, offering language support, themes, and formatting options. Code written in the editor is temporarily stored in React state and automatically saved to the file manager's structure.

The terminal interface, built using Xterm.js, connects to the backend via a WebSocket channel. Input from the terminal is captured and transmitted to Node-PTY, which processes the commands and returns output that is rendered in real time.

B. Backend Implementation

The backend is designed to be lightweight and modular. The main server file initializes Express routes for:

- `/execute` — Handles Judge0 API requests.
- `/terminal` — Manages WebSocket connections for live terminals.
- `/file` — Supports file operations (create, delete, rename, read, write).

A simplified snippet of terminal setup is shown below:

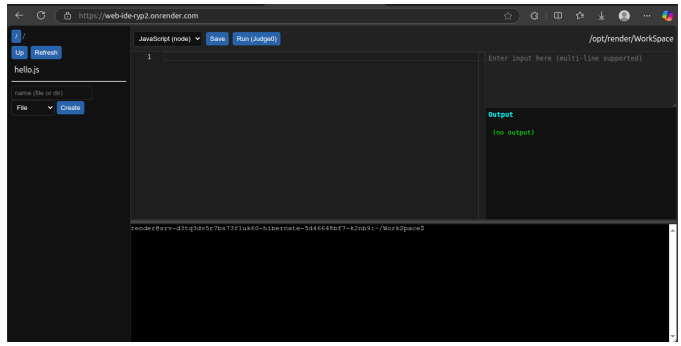

Fig. 2. Complete Interface: Monaco Editor, File Manager, and Terminal

Listing 1. Node-PTY Terminal Setup

```
const pty = require("node-pty");
const os = require("os");
const shell = os.platform() === "win32" ? "powersh

const terminal = pty.spawn(shell, [], {
  name: "xterm-color",
  cols: 80,
  rows: 24,
  cwd: process.env.HOME,
  env: process.env
});

terminal.onData((data) => {
  socket.emit("output", data);
});
```

This setup allows users to interact directly with a live shell through the browser.

V. Results and Usability Evaluation

The system was tested for functionality, interface consistency, and usability. Tests were conducted using modern browsers (Chrome, Firefox, and Edge).

A. Functional Results

- The compiler executed multiple programming languages successfully via Judge0.
- The terminal displayed real-time outputs and accepted user commands.
- The file manager supported creation and organization of multiple source files.

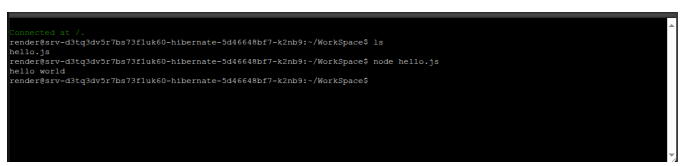

Fig. 3. Working Terminal with Real-Time Command Execution

B. User Experience

User feedback focused on simplicity, interactivity, and design clarity. The terminal provided a strong sense of realism, while the integrated output from Judge0 eliminated the need for separate execution windows. The responsive layout ensured a consistent experience across devices.

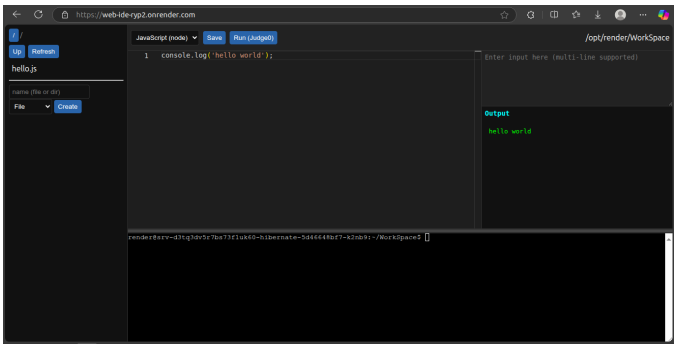

Fig. 4. Code Execution Output via Judge0 API

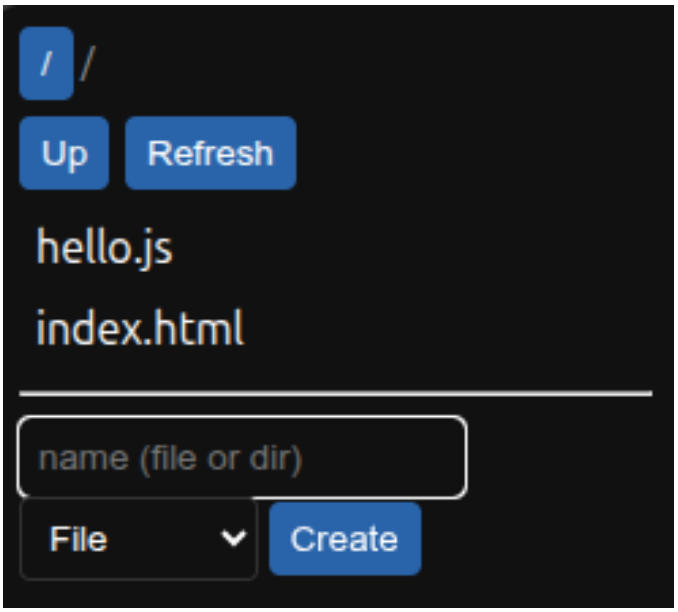

Fig. 5. Inbuilt File Management System for Project Organization

C. Usability Focus

Rather than optimizing performance metrics, the project emphasizes **user usability and experience**. The design supports:

- Clear navigation and visual hierarchy.
- Smooth transitions between code editing and terminal use.
- Logical file organization for small-scale projects.

VI. Discussion

The combination of Judge0, Xterm.js, and Node-PTY provides a versatile solution for online code execution. This setup allows a unique blend of real-time compilation and interactive shell operations, replicating a local coding experience.

While the current version supports single-user sessions, future development could integrate:

- **Collaboration:** Multiple users editing and executing in real time.
- **Persistent Storage:** Saving user projects to a cloud database.
- **Authentication:** Secure login and personalized workspace.
- **Git Integration:** Direct commit and pull functionality from the terminal.

These enhancements would move the project closer to a complete, cloud-based IDE system.

VII. Conclusion

The Online Code Compiler successfully integrates a modern editor, real-time terminal, and project file manager into one unified system. Built entirely using web technologies, it demonstrates how powerful browser-based environments can now replicate desktop IDE functionality. The focus on usability and interactivity makes it a strong platform for programming education, testing, and lightweight development.

Future work will explore cloud persistence, user authentication, and shared project spaces, expanding its use from individual practice to collaborative software engineering.

References

[1] Judge0 API Documentation, *https://judge0.com*
[2] Monaco Editor, Microsoft Docs, *https://microsoft.github.io/monaco-editor*
[3] Xterm.js Official Documentation, *https://xtermjs.org*
[4] Node-PTY GitHub Repository, *https://github.com/microsoft/node-pty*
[5] React Documentation, *https://react.dev*
[6] Node.js Official Documentation, *https://nodejs.org*